UNIVERSIDAD LIBRE

Facultad de Ingeniería

Taller 7: Interfaces gráficas con JFrame
Sistema de gestión de envíos

Programación 1 — Entrega del taller

Docente: Andrés Castaño

Integrantes del grupo

N.º	Nombre completo	Código estudiantil
1	[Nombre del integrante 1]	[código]
2	[Nombre del integrante 2]	[código]
3	[Nombre del integrante 3]	[código]
4	[Nombre del integrante 4 – si aplica]	[código]

Repositorio: [URL del repositorio Git] **Rama de entrega:** entregable

1. Descripción general

La aplicación permite registrar envíos de una empresa de mensajería en tres modalidades (**Estándar**, **Express** e **Internacional**) y calcula el costo de cada uno. Todas las modalidades implementan la interfaz `Envio`, por lo que el programa las trata de forma uniforme dentro de un `ArrayList<Envio>` (polimorfismo). La interfaz gráfica es un `JFrame` que cambia dinámicamente el campo adicional según la modalidad elegida.

Se siguió la misma lógica del ejemplo *Usuarios y notas*: una capa de lógica (clases del dominio + una clase `Registros` que guarda la colección) separada de la capa de presentación (JFrame), con un método `actualizarTabla()` que vuelca la colección al `DefaultTableModel`.

2. Arquitectura

Paquete	Clase	Responsabilidad
logica	Envio (interfaz)	Declara calcularCosto() y los getters comunes (código, destinatario, peso, tipo, dato adicional).
logica	EnvioEstandar	Atributo propio: dirección. Costo = peso × $5.000.

Paquete	Clase	Responsabilidad
logica	EnvioExpress	Atributo propio: hora límite. Costo = peso × $5.000 + $15.000.
logica	EnvioInternacional	Atributo propio: país destino. Costo = base + 10 % impuesto + 5 % seguro, con base = peso × $12.000.
logica	Registros	Guarda el ArrayList<Envio>; agregarEnvio, existeCodigo, obtenerEnvios, calcularTotal.
IGU	IGEnvios (JFrame)	Formulario, validaciones, creación del objeto según la modalidad y JTable.

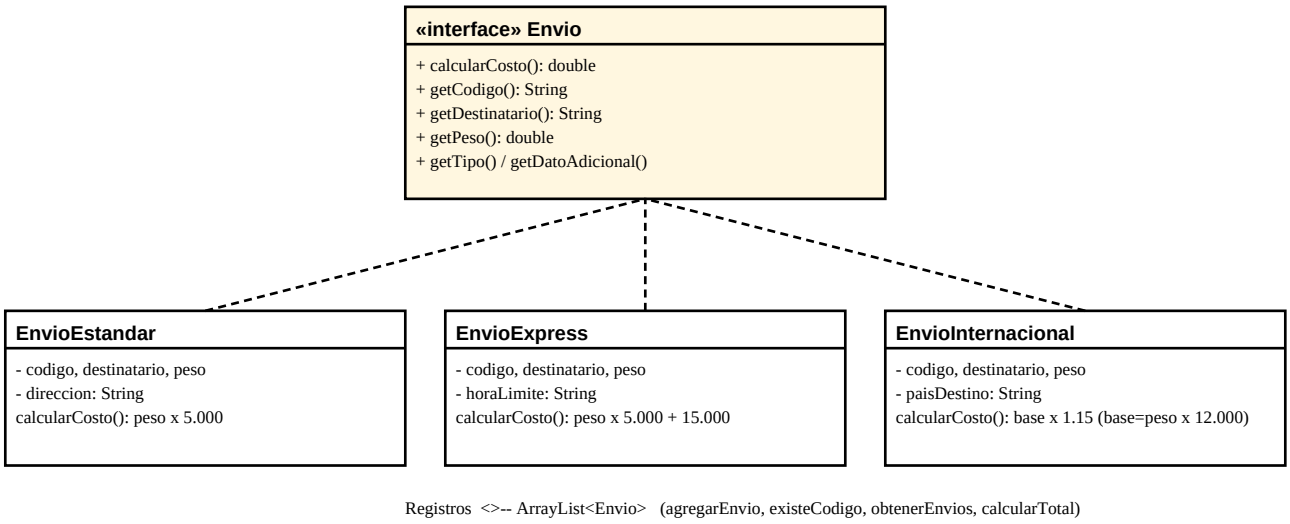

Figura 1. Diagrama de clases simplificado.

3. Cómo está construida la interfaz

La ventana usa `BorderLayout`: arriba el panel del formulario (`GridBagLayout`) y en el centro la tabla dentro de un `JScrollPane`. Cada componente tiene un nombre (`setName`) para identificarlo fácilmente.

Componente	Nombre	Función
JComboBox	comboModalidad	Elige Estándar, Express o Internacional.
JTextField ×3	campoCodigo, campoDestinatario, campoPeso	Campos comunes, siempre visibles.
JLabel + JTextField	campoDireccion	Solo visible en Estándar.
JLabel + JTextField	campoHoraLimite	Solo visible en Express.
JLabel + JTextField	campoPais	Solo visible en Internacional.
JButton	btnRegistrar	Crea el envío, lo agrega a la colección y confirma con JOptionPane.
JTable + DefaultTableModel	tablaEnvios	Lista código, tipo, destinatario, peso, dato adicional y costo (no editable).

Figura 2. Esquema del formulario en cada modalidad (el campo resaltado es el único adicional visible). Sustituir/complementar con capturas reales de la aplicación.

3.1 Campos dinámicos (JComboBox)

El `ActionListener` del combo llama a `actualizarCamposVisibles()`, que aplica `setVisible()` a la etiqueta y al campo de cada modalidad (misma técnica del JCheckBox). Las tres parejas ocupan la misma fila del formulario, así que al ocultarse no dejan huecos:

```
private void actualizarCamposVisibles() {
    String modalidad = (String) comboModalidad.getSelectedItem();
    boolean estandar = ESTANDAR.equals(modalidad);
    boolean express = EXPRESS.equals(modalidad);
    boolean internacional = INTERNACIONAL.equals(modalidad);

    etiquetaDireccion.setVisible(estandar);   campoDireccion.setVisible(estandar);
    etiquetaHoraLimite.setVisible(express);   campoHoraLimite.setVisible(express);
    etiquetaPais.setVisible(internacional);   campoPais.setVisible(internacional);
    getContentPane().revalidate();  getContentPane().repaint();
}
```

3.2 Botón Registrar

Al pulsar **Registrar**: (1) se leen y validan los campos comunes; (2) según la modalidad se lee el campo adicional y se crea el objeto `EnvioEstandar`, `EnvioExpress` o `EnvioInternacional`, guardado en una variable de tipo `Envio`; (3) se rechaza si el código ya existe; (4) se agrega a `Registros`; (5) se actualiza la tabla; (6) se muestra un `JOptionPane` con el código, el tipo y el costo; (7) se limpian los campos.

```
Envio envio;
if (modalidad.equals(ESTANDAR)) {
    envio = new EnvioEstandar(codigo, destinatario, peso, direccion);
} else if (modalidad.equals(EXPRESS)) {
    envio = new EnvioExpress(codigo, destinatario, peso, horaLimite);
} else {
    envio = new EnvioInternacional(codigo, destinatario, peso, pais);
}
this.registros.agregarEnvio(envio);
this.actualizarTabla();          // vuelca la colección al DefaultTableModel
```

3.3 Validaciones

Situación	Respuesta de la aplicación
Código, destinatario o peso vacíos	Advertencia «Complete el código, el destinatario y el peso.»
Peso no numérico o ≤ 0	Advertencia; no se registra (se acepta coma o punto decimal).
Campo adicional vacío	Advertencia indicando cuál campo falta.
Código ya registrado	Advertencia; no se registra el duplicado.

4. Cómo consumir el servicio

4.1 Desde la interfaz (uso normal)

1) Ejecutar `IGU.IGEnvios` (NetBeans: clic derecho → *Run File*; Maven: `mvn compile exec:java -Dexec.mainClass=IGU.IGEnvios`). 2) Elegir la modalidad en el combo. 3) Diligenciar código, destinatario, peso y el campo adicional que aparece. 4) Pulsar **Registrar**: aparece la confirmación con el costo y el envío se añade a la tabla.

4.2 Desde código (capa de lógica)

La lógica no depende de Swing, por lo que puede reutilizarse desde cualquier otra interfaz o prueba:

```
Registros registros = new Registros();
registros.agregarEnvio(new EnvioEstandar("ENV001", "Laura", 4, "Calle 45 # 10-20"));
registros.agregarEnvio(new EnvioInternacional("ENV005", "Felipe", 5, "España"));

for (Envio e : registros.obtenerEnvios()) {          // trato uniforme (polimorfismo)
    System.out.println(e.getCodigo() + " -> " + e.calcularCosto());
}
```

5. Pruebas con los datos sugeridos

Los seis envíos se registran desde el formulario. Costo esperado (verificado ejecutando las clases de la capa de lógica):

Código	Destinatario	Peso	Tipo	Cálculo	Costo en tabla
ENV001	Laura	4 kg	Estándar	4 × 5.000	$ 20.000
ENV002	Carlos	8 kg	Estándar	8 × 5.000	$ 40.000
ENV003	Andrés	3 kg	Express	3 × 5.000 + 15.000	$ 30.000
ENV004	María	10 kg	Express	10 × 5.000 + 15.000	$ 65.000
ENV005	Felipe	5 kg	Internacional	60.000 + 6.000 + 3.000	$ 69.000
ENV006	Diana	12 kg	Internacional	144.000 + 14.400 + 7.200	$ 165.600

Capturas de la aplicación: [insertar aquí las capturas de la interfaz con los seis envíos registrados y del mensaje de confirmación]

6. Entrega en Git

El trabajo se entrega en la rama **entregable**. Esa rama no debe recibir commits posteriores a la fecha de entrega.

```
git init
git checkout -b entregable
git add .
git commit -m "Taller 7: sistema de gestion de envios"
git remote add origin <URL_DEL_REPOSITORIO>
git push -u origin entregable
git log --format="%h %ad %s" --date=short     # verificar fechas
```